DRONE SECURITY SYSTEM

AI-Powered Surveillance & Threat Detection

Project Report

YOLOv8s | OpenCLIP | LangGraph | Nova Lite | Claude Haiku | ChromaDB | SQLite | Streamlit | ByteTrack | Moondream2

Generated: 20 April 2026

TABLE OF CONTENTS

1. Executive Summary

The Drone Security System is an AI-powered, multi-modal security monitoring platform that combines real-time computer vision, large language models, and agentic decision-making to detect, assess, and respond to security threats captured by a drone camera. The system processes raw video frames through a six-stage LangGraph pipeline -- perception, contextualisation, rule evaluation, LLM escalation, alerting, and logging -- and exposes the results through an interactive Streamlit dashboard with a natural-language query interface.

The platform is designed for property owners and security operators who need automated 24/7 perimeter monitoring without requiring a dedicated human analyst watching a live feed. Key outcomes include sub-5-second alert latency for rule-based threats, LLM-assisted false-positive reduction, and natural-language querying of the full indexed footage archive.

2. Problem Statement

Traditional CCTV-based security systems generate continuous footage that is rarely reviewed until after an incident occurs. Human operators experience alert fatigue when systems fire on every motion event, and retrospective forensic search through hours of footage is time-consuming and error-prone.

The specific challenges this system addresses are:

* Real-time detection of meaningful security events (intrusion, loitering, vehicle anomalies) with a low false-positive rate.
* Contextual reasoning to distinguish genuine threats from benign activity (e.g., authorised staff after hours vs. intruder).
* Efficient retrospective querying using natural language rather than timestamp scrubbing.
* Drone telemetry awareness -- battery, altitude, and flight mode data must be monitored alongside visual feeds so that drone health events (low battery, altitude anomaly) are treated as first-class security signals.

The system assumes the drone operates as a stationary overhead camera hovering at a fixed GPS position. Zone labels (e.g., main_gate, perimeter_north) are static for a session. If the drone is airborne and changes location, a GPS-to-zone geofence mapping layer would be required -- this is documented as a known limitation and future extension.

3. System Architecture

The architecture follows a layered pipeline model with four functional layers:

[Layer 1 -- Ingestion]

VideoIngestor reads from a local video file, an RTSP stream, or a text-fallback JSONL file. It applies frame sampling (every N-th frame), optional resize, and attaches a TelemetrySimulator snapshot to every FramePacket. FramePreprocessor standardises the image to the three resolutions required downstream: 640px for YOLO, 378px for the VLM, and 224px for CLIP.

[Layer 2 -- Processing (LangGraph)]

Six typed nodes execute in sequence with conditional routing:

* perceive -- YOLO detection, CLIP embedding, VLM caption (async)
* contextualize -- track durations, vehicle counts, telemetry summary
* rule_check -- evaluate all YAML rules against current state
* llm_judge -- (conditional) Claude Haiku reviews ambiguous rule hits
* alert -- store events and alerts in SQLite with cooldown dedup
* log -- persist frame, objects, and CLIP embedding

[Layer 3 -- Storage]

SQLite stores structured data across four tables: frames, objects, events, alerts. ChromaDB stores CLIP embeddings with zone/class/caption metadata for semantic search. HybridRetriever routes queries: temporal/class queries go to SQL; open-ended semantic queries go through ChromaDB.

[Layer 4 -- Delivery]

The Streamlit dashboard provides three tabs: Live Feed (real-time video + detections + telemetry + alerts), Timeline (filterable event log with evidence frames), and Ask (natural-language RAG chatbot over the indexed footage).

4. AI Tools and Models

4.1 YOLOv8s + ByteTrack

Object detection runs YOLOv8s (the 'small' variant at ~22M parameters) via the Ultralytics library. ByteTrack is enabled through model.track(), assigning persistent track IDs across frames. This is essential for loitering detection, which requires knowing that person in frame 42 is the same individual as person in frame 120. YOLOv8s processes a 640px frame in approximately 20-50ms on a modern CPU.

4.2 Amazon Bedrock -- Nova Lite (VLM Captioner)

After YOLO detection, the frame is sent to Amazon Nova Lite via Amazon Bedrock for a 25-word security-focused caption. The caption describes behavioural context that bounding boxes alone cannot convey (e.g., 'person crouching near fence, appears to be examining the gate latch'). This enables caption-based rules such as gate_tampering_attempt and suspicious_crouching. The VLM call runs asynchronously in a ThreadPoolExecutor so the hot path (YOLO + rules) is never blocked by the ~500ms Bedrock latency.

4.3 Amazon Bedrock -- Claude Haiku (LLM Judge)

When a rule is flagged needs_llm: true, the llm_judge node invokes Claude Haiku with the current context summary, caption, and the triggered rule hits. The LLM returns a structured verdict (genuine / false_positive) and an alert message. This reduces false positives for broad rules (e.g., crowd_gathering) without requiring per-zone threshold tuning.

4.4 OpenCLIP ViT-B/32

Every processed frame receives a 512-dimensional CLIP embedding from the OpenCLIP ViT-B/32 model. These embeddings are stored in ChromaDB and support natural-language visual search: a text query is also embedded by CLIP, and cosine similarity retrieves the visually closest frames. This powers the Ask tab's semantic search path for queries like 'blue truck near the main gate'.

4.5 Moondream2 (Local VLM Fallback)

If Bedrock credentials are unavailable, VLMCaptioner falls back to Moondream2, a 2GB local vision-language model. It runs on CPU at approximately 1-2fps -- slow, but functional for offline or development environments.

4.6 LangGraph

LangGraph orchestrates the six-node pipeline as a typed StateGraph. Conditional edges route frames based on needs_llm and rule_hits flags. Each node is a pure function (state dict in, partial dict out), making the pipeline independently testable and straightforward to extend with new branches.

5. Telemetry Pipeline

A TelemetrySimulator generates physically plausible drone telemetry for the full duration of a patrol flight. The simulator is anchored to a start timestamp and computes interpolated values for any requested datetime via get_telemetry(ts).

Fields produced: battery_pct, alt_m, lat, lon, speed_ms, heading_deg, flight_mode (TAKEOFF / PATROL / HOVER / RETURN / LANDING), signal_strength, temp_c, vspeed_ms, satellites.

Four named scenarios are provided:

* perimeter_patrol -- standard 35m cruise, 80m radius
* night_watch -- low-altitude slow patrol, 20m cruise
* emergency_response -- fast high-altitude response, 60m cruise
* battery_low_test -- starts at 25% to trigger low-battery rules quickly

The telemetry snapshot is injected into every FramePacket by VideoIngestor, propagated through AgentState, stored as a JSON blob in the SQLite frames table, surfaced in HybridRetriever search results, and displayed in the dashboard Live tab as a real-time metrics strip (battery, altitude, speed, flight mode, GPS, signal).

Three telemetry rules are active in rules.yaml:

* battery_critical -- battery < 10% (high severity)
* low_battery -- battery < 20% (medium severity)
* altitude_anomaly_high -- altitude > 120m (medium severity)
* altitude_anomaly_low -- altitude < 5m (medium severity)

6. Rule Engine

All security rules are declared in configs/rules.yaml. The RuleEngine class parses this file and evaluates every rule against the current AgentState on each frame. Rules are hot-reloadable -- engine.reload() atomically replaces the active rule set without restarting the pipeline.

Rule categories:

Person-based:
after_hours_person, after_hours_zone_entry, loitering (>30s in zone), crowd_gathering (>=3 people), tailgating

Vehicle-based:
after_hours_vehicle, vehicle_repeat_entry, stopped_vehicle, unattended_vehicle, convoy_detection

Zone / perimeter:
forbidden_zone, perimeter_breach

Caption-based (requires VLM):
gate_tampering_attempt, coordinated_breach_attempt, lookout_behavior, tool_use_near_infrastructure, suspicious_crouching, perimeter_climbing_attempt, forced_entry_attempt, suspicious_group_activity

Telemetry:
battery_critical, low_battery, altitude_anomaly_high, altitude_anomaly_low

Rules marked needs_llm: true route to the llm_judge node for contextual review before an alert is stored. Alerts are deduplicated with a 5-minute cooldown per (rule_name, zone) pair to prevent alert-spam during continuous events.

7. Memory and Retrieval

7.1 SQLite (Structured Store)

Four tables: frames (id, ts, video_id, frame_index, frame_path, caption, telemetry_json, zone), objects (frame_id, class, confidence, bbox, track_id), events (id, start_ts, type, severity, description, frame_ids, zone), alerts (id, event_id, ts, channel, message, acked).

Indexes on ts and zone enable fast time-window queries. WAL mode allows concurrent dashboard reads during pipeline writes.

7.2 ChromaDB (Vector Store)

CLIP embeddings are stored with metadata (zone, class_names, caption, ts, video_id). The HNSW index supports approximate nearest-neighbour search at sub-millisecond latency for collections up to ~1M vectors.

7.3 HybridRetriever

Query routing:

* Temporal / class queries (has_time=True or class_filter set): SQL-first via temporal_search() -- deterministic, 100% recall within the time window.
* Semantic queries: CLIP-first via search() -- over-fetches 2K candidates from ChromaDB, enriches with SQLite metadata, re-ranks by cosine similarity.

The Ask tab uses an LLM intent parser to extract structured fields (has_time, start, end, class_filter, zone, has_telemetry, battery_below, altitude_above, flight_mode_filter) from the natural-language query before routing to the appropriate retrieval path.

8. Dashboard

The Streamlit dashboard has three tabs:

Live Feed

Processes a selected video file or RTSP stream in real time. Displays annotated frames with YOLO bounding boxes and track IDs, a VLM caption strip, frame/detection/alert/FPS metrics, a colour-coded telemetry panel (battery, altitude, speed, flight mode, GPS, signal), and a live alert feed showing the most recent 15 alerts.

Timeline

Filterable event log from SQLite. Filters: event type (all 20+ types including telemetry events), severity, zone, and hours-back window. Each event expands to show the description, evidence frames (JPEG thumbnails), and associated alerts with one-click acknowledgement. LLM-flagged false positives are tagged with a review warning.

Ask

Multi-turn RAG chatbot. The user types a question; the system parses intent, routes to the correct retrieval path, builds a context string (frames + telemetry + events), and calls Claude Haiku via Bedrock for a factual answer. Source frames are shown below each answer with image thumbnails, timestamps, zone, confidence score, and telemetry snapshot. Suggested follow-up questions are generated automatically after each answer.

9. Testing

The test suite contains 124 tests across three layers:

Unit tests (tests/unit/):
test_sqlite_store.py -- 42 tests covering all CRUD operations, time-window queries, vehicle counts, alert acknowledgement, and telemetry JSON round-trip.
test_hybrid_retriever.py -- 27 tests covering semantic search, temporal search, SQL fallback, class/zone filtering, events_summary, and stats.

Integration tests (tests/integration/):
test_agent_pipeline.py -- 55 tests exercising the full LangGraph graph with stub models (StubDetector, StubCaptioner, StubEmbedder). Covers perceive -> log for clean frames, rule hits without LLM, LLM genuine verdict, LLM false-positive verdict, loitering accumulation (two-invoke pattern), cooldown deduplication, and telemetry propagation through state.

Key testing decisions:

* SQLite :memory: uses a cached single connection (self._mem_conn) to avoid the 'no such table' error that arises when each query opens a fresh empty DB.
* ChatBedrock is patched before build_agent() is called, because the LLM is instantiated at graph-build time, not at invoke time.
* Loitering tests use a two-invoke pattern: first call seeds the track, second call (65s later) triggers the >30s threshold.

10. Key Design Decisions

1. LangGraph for agent orchestration -- conditional routing as a first-class concept; typed AgentState makes inter-node contracts explicit; each node is independently unit-testable.

2. Async VLM -- VLM caption submitted to a background ThreadPoolExecutor; hot path (YOLO + rules) runs at 12-20fps unblocked; caption-based rules accept a ~500ms lag.

3. Declarative YAML rules -- operator-editable without code changes; hot-reloadable during live incidents; needs_llm flag selectively gates on Claude judgment.

4. Hybrid retrieval (SQL + ChromaDB) -- SQL for exact temporal/class queries (100% recall); CLIP for open-ended semantic queries; LLM intent parser routes between them at query time.

5. Frame storage separation -- model inputs use full source resolution; saved JPEG thumbnails are compressed at a configurable preset (default 640x360 / q75) to manage disk usage without affecting detection quality.

6. Stationary drone assumption -- zone labels are static per session. GPS coordinates from TelemetrySimulator are stored for operator awareness but do not drive zone assignment. A geofence polygon mapping layer (e.g., using shapely) would be required for a mobile drone covering multiple zones in a single flight.

7. Alert deduplication cooldown -- 5-minute per-(rule, zone) cooldown prevents alert spam during continuous events while ensuring new incidents are reported.

8. Two-tier VLM -- Bedrock Nova Lite primary for sub-500ms cloud captioning; Moondream2 local fallback for offline/no-credential environments.

11. Assumptions and Limitations

Stationary drone deployment

The system is validated for a drone hovering at a fixed position. Zone names are assigned at session start and remain constant. Extending to a mobile drone requires GPS polygon geofencing and per-frame zone lookup.

Telemetry simulation

In the absence of a real drone SDK, telemetry is generated by TelemetrySimulator using a physics-based model (Kalman-style phase transitions, circular GPS orbit, battery discharge curve). A production deployment would replace the simulator with a real MAVLink or DJI SDK telemetry stream.

AWS credentials required for cloud VLM

The Nova Lite captioner and Claude Haiku judge both require valid AWS credentials with Bedrock model access. The system falls back to Moondream2 locally if credentials are absent, but caption quality is lower and throughput drops to ~1fps.

Single-camera, single-zone per session

The current pipeline processes one video source per session. Multi-camera fusion (correlating the same event across two camera angles) is not implemented.

No re-identification across sessions

ByteTrack assigns track IDs within a session. A person who leaves and re-enters the frame is assigned a new track ID. Cross-session identity (same individual on different days) is not implemented.

12. Future Work

* GPS geofencing for mobile drone deployment (shapely polygon-in-point)
* Real drone telemetry integration via MAVLink / DJI Mobile SDK
* Multi-camera fusion and cross-view event correlation
* Re-identification across sessions using face/gait embeddings
* Active learning loop: operator corrections fed back to rule tuning
* Exportable incident report generation (PDF per event with evidence frames)
* Push notification channel (email / SMS / Slack) via alert dispatcher
* Containerised deployment with Docker Compose for one-command setup
* Quantised local LLM judge (e.g., Phi-3-mini) to remove Bedrock dependency for the false-positive review path